Parallelism Project: Is there parallelism among us?

Fork/Join Performance Analysis

Tibo Stans[1]
[1]Vrije Universiteit Brussel
tibo.stans@vub.be

Abstract

This report presents the design, implementation, and performance evaluation of parallel algorithms developed to detect "Amongi" characters within large-scale images, specifically sourced from the Reddit r/place collaborative dataset. The core of this project lies in the application of the Java Fork/Join framework to effectively parallelize the computationally intensive image analysis task. My implementation strategy involved two primary phases: an initial parallelization of the x-axis traversal, exploring both global `ConcurrentHashMap` and local `HashMap` strategies for result aggregation, followed by an extension of the local `HashMap` approach to incorporate y-axis parallelism, controlled by a configurable sequential threshold. I conducted a comprehensive performance evaluation of these implementations on two distinct systems: my local Ryzen 5800X desktop and the Firefly high-performance computing server (an AMD EPYC based system). This analysis covers critical aspects such as overhead, speed-up, scalability, and the influence of the sequential threshold parameter, directly addressing the questions outlined in the project specification for both platforms. The findings demonstrate substantial performance gains through parallel execution, although optimal efficiency is contingent on the chosen parallelization strategy, parameter tuning, and the specific hardware architecture. Notably, diminishing returns were observed when utilizing logical threads beyond physical core counts via Simultaneous Multithreading (SMT). This report also includes a theoretical examination of the work, span, and task creation characteristics inherent in the developed parallel solutions.

1 Implementation

To address the challenge of efficiently finding Amongi in large images, I parallelized the provided sequential finder by leveraging the Java Fork/Join framework. My implementation strategy was structured into two distinct phases, focusing on the `ParallelFinderGlobalHashMap` and `ParallelFinderSubtotals` classes.

1.1 Phase 1: Parallelising the x-axis

In this initial phase, I parallelized the traversal over the x-coordinates of the input image, while processing the y-coordinates sequentially within each x-coordinate subtask. The classes implementing this phase are:

- **`ParallelFinderSubtotals` (Phase 1 configuration):** Implemented in `solutions.ParallelFinderSubtotals.java`. This class, when configured for Phase 1, utilizes its inner class `XFinderTask` to divide the x-coordinate range. Each task computes subtotals in a local `java.util.HashMap<Integer, Integer>`. The y-axis sequential threshold parameter is set to `Integer.MAX_VALUE`.
- **`ParallelFinderGlobalHashMap`:** Implemented in `solutions.ParallelFinderGlobalHashMap.java`. This class uses its inner class `FinderTask` to manage the division of work along the x-axis, with tasks updating a single, shared `java.util.concurrent.ConcurrentHashMap<Integer, Integer>`.

1.2 Phase 2: Parallelising the y-axis

Building upon the local HashMaps strategy, this phase, also implemented within `solutions.ParallelFinderSubtotals.java`, extended the parallelization to include the y-coordinates.

- **`ParallelFinderSubtotals` (Phase 2 configuration):** The `XFinderTask` further delegates y-axis processing to instances of `YFinderTask`. These tasks recursively divide the y-axis segments based on a configurable sequential cut-off threshold T.

1.3 Fork/Join Patterns Used

The predominant Fork/Join pattern I employed throughout both implementation phases is **Divide and Conquer**.

- For parallelizing the x-axis (in both `ParallelFinderGlobalHashMap` and `ParallelFinderSubtotals`), the image's width is recursively partitioned. This can be seen as

a form of parallel map operation on segments of the x-axis.

- In the `ParallelFinderSubtotals` implementation, the merging of local `HashMaps` embodies a **Reduction** operation, where partial results are combined.
- During Phase 2, my `ParallelFinderSubtotals` implementation applies the Divide and Conquer strategy hierarchically.

Other specific patterns like prefix sum, pack, or parallel sorting were not directly applicable or implemented for this problem. Java lambdas were consciously avoided within the Fork/Join tasks as per project guidelines.

2 Evaluation

I evaluated the performance of the implemented parallel solutions using benchmark results obtained from the `benchmark_results.txt` file (for Firefly) and data provided separately for my local machine. Benchmarks were conducted on both my local Ryzen 5800X system and the Firefly server.

2.1 Experiments and Experimental Set-up

The experimental setup for both systems was configured as follows:

2.1.1 Local Benchmarking System (Ryzen 5800X)

- Hardware:
  - CPU: AMD Ryzen 7 5800X (8 cores, 16 threads via SMT)
  - RAM: 32 GB DDR4 @ 3200MHz (Corsair)
  - GPU: NVIDIA GeForce RTX 3090 (Not directly used for this CPU-bound task)
  - Storage: Samsung 970 EVO M.2 NVMe SSD 1TB
- Software:
  - OS: Ubuntu 24.04.2 LTS
  - Java: OpenJDK version 17 LTS

2.1.2 Firefly Server Benchmarking System

- Hostname: `firefly.vub.ac.be`
- Hardware:
  - Model: Custom built (Year: 2021)
  - CPU: AMD Ryzen™ Threadripper™ 3990X Processor (64 cores @ 2.9GHz base, 4.3GHz boost, 128 threads via SMT)
  - RAM: 128 GB DDR4-3200
  - SSD: 2x2TB NVMe
- Software:
  - OS: Ubuntu 24.04.2 LTS
  - Java: OpenJDK version 14.0.2 (Java 14)

2.1.3 Common Benchmarking Methodology

- **Benchmarking Tool:** The benchmark results were generated using the custom `BenchmarkMain.java` script. As per the "Guidelines for Performance Evaluation," each algorithm configuration was subjected to 3 warm-up iterations followed by 5 measurement iterations. The average of these 5 measurements is reported.
- **Dataset:** The evaluation utilized several PNG image files, with a primary focus on `images/place_23k_23k.png` (23000x23000 pixels) for detailed analysis.
- **Metrics Measured:** Average Execution Time (ms), Maximum Time (ms), Minimum Time (ms), Standard Deviation (ms), and Speedup (T_s/T_p). T_s is the average execution time of the baseline sequential `SequentialFinder` on the respective machine.
- **JVM Heap Size:** Default JVM heap sizes were used for all reported benchmarks on both systems.

2.2 Overhead and Speed-up (Phase 1 Implementations)

This section discusses the overhead and speed-up of the Phase 1 implementations.

2.2.1 What is the overhead of your implementations? Is this overhead acceptable? What causes this overhead?

Overhead is $T_p(1) - T_s = T_p(1) \times (1 - S_p(1))$.

On Local Ryzen 5800X (for `images/place_23k_23k.png`):

- **PGH:** $T_p(1) = 40690.96$ ms, $S_p(1) = 0.99$. Overhead ≈ 406.91 ms ($\approx 1\%$).
- **PST (Y sequential):** $T_p(1) = 40727.64$ ms, $S_p(1) = 0.99$. Overhead ≈ 407.28 ms ($\approx 1\%$).

On my local machine, both implementations show a small, comparable overhead of about 1% when run on a single core. This is acceptable.

On Firefly Server (for `images/place_23k_23k.png`):

- **PGH:** $T_p(1) = 62529.61$ ms, $S_p(1) = 0.99$. Overhead ≈ 625.30 ms ($\approx 1\%$).
- **PST (Y sequential):** $T_p(1) = 62361.32$ ms, $S_p(1) = 1.00$. Overhead ≈ 0 ms.

On Firefly, PGH shows a similar relative overhead, while PST (Y sequential) has negligible overhead. The causes of overhead are consistent across systems: task management (creation, scheduling, joining), synchronization (especially for PGH's `ConcurrentHashMap`), and Fork/Join pool initialization.

2.2.2 What are the speed-ups of your implementations? Are these speed-ups higher or lower than expected? Why?

On Local Ryzen 5800X (for `images/place_23k_23k.png`, see Figure 1):

- **PGH:** Max speed-up of $7.23\times$ at 16 threads.

- **PST (Y sequential):** Max speed-up of $7.25\times$ at 16 threads.

Given 8 physical cores, an ideal speed-up would be $8\times$. The achieved $\approx 7.2\times$ using 16 threads (utilizing SMT) is quite good, indicating efficient use of the available cores and some benefit from SMT. It's slightly lower than $8\times$, which is expected due to Amdahl's Law, memory contention, and SMT not typically doubling performance.

On Firefly Server (for `images/place_23k_23k.png`, see Figure 2):

- **PGH:** Max speed-up of $31.18\times$ at 64 cores.
- **PST (Y sequential):** Max speed-up of $31.17\times$ at 64 cores.

With 64 physical cores, the achieved $\approx 31\times$ speed-up is substantial but less than half the ideal linear speed-up. This is lower than a highly optimistic expectation but reasonable for such a large-scale parallel system due to increased impact of memory bandwidth limits, NUMA effects, and the sequential fraction of the code. The drop at 128 threads is due to SMT limitations.

Reasons for sub-ideal speed-ups are common: Amdahl's Law, memory bandwidth saturation, task overheads, load imbalance, and synchronization.

2.2.3 What speed-up is larger: the computational or application speed-up? Is this what you expect? Why?

The reported T_s/T_p is effectively the application speed-up. A theoretical "computational speed-up" (ignoring all system overheads) would always be higher. I expect the measured application speed-up to be lower due to these real-world factors, which is consistent with the results on both systems.

2.2.4 How well do your implementations scale? Why do they (not) scale well?

On Local Ryzen 5800X (see Figure 1): Both implementations scale well up to 8 cores/threads. Performance continues to increase up to 16 threads, indicating effective use of SMT, though the incremental gain from 8 to 16 threads is less than from 4 to 8. This suggests good scalability within the physical core count and some additional benefit from SMT.

On Firefly Server (see Figure 2): Scaling is good up to 64 physical cores, with diminishing returns as core count increases. Beyond 64 cores (using 128 threads via SMT), performance degrades. This indicates that for this problem, SMT on the Threadripper does not provide further speed-up and may introduce detrimental contention.

The reasons for non-perfect scaling are consistent: Amdahl's Law, memory contention (more pronounced on Firefly due to more cores), and task overheads. SMT limitations are evident on both systems at their respective maximum thread counts.

2.2.5 Based on your experiments, which of both implementations is the best? Why?

On Local Ryzen 5800X: Both PGH and PST (Y sequential) perform very similarly, with PST achieving a marginally higher peak speed-up ($7.25\times$ vs $7.23\times$). Given the near-identical performance, either could be chosen, but PST might be slightly preferred for its conceptual simplicity.

On Firefly Server: PST (Y sequential) had negligible single-core overhead compared to PGH's small overhead. Peak speed-ups were almost identical. Again, PST (Y sequential) is marginally preferable due to lower single-core overhead.

Overall, **`ParallelFinderSubtotals` (Y sequential) appears to be slightly better or equivalent** on both systems.

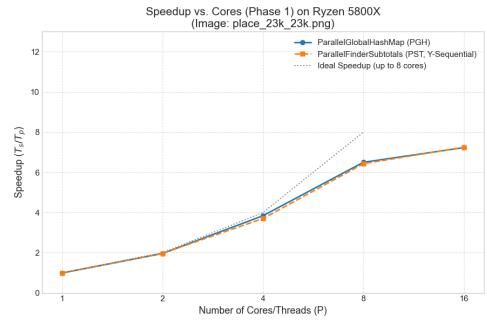

Figure 1: Speedup vs. Cores (P) for Phase 1 on Ryzen 5800X (`images/place_23k_23k.png`).

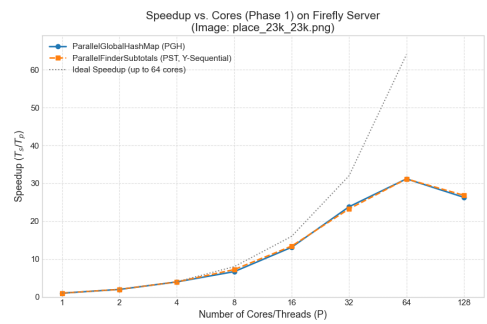

Figure 2: Speedup vs. Cores (P) for Phase 1 on Firefly Server (`images/place_23k_23k.png`).

2.3 Optimal Threshold (Phase 2 - ParallelFinderSubtotals Parallel Y)

This section investigates the influence of the y-axis sequential threshold T for the `ParallelFinderSubtotals` implementation.

2.3.1 How does the threshold influence the overhead and speed-up of your implementations? What happens when you make the threshold larger or smaller? Explain.

On Local Ryzen 5800X (P=16 threads, for `images/place_23k_23k.png`, see Figure 3):

- Small T (e.g., $T = 1$): Speed-up is low ($2.85\times$). Excessive task creation overhead dominates.
- Increasing T: Speed-up improves significantly, peaking around $T = 200$ ($7.16\times$). Task granularity becomes more optimal.
- Large T (e.g., $T = 500$): Speed-up slightly decreases ($7.13\times$). Tasks become too coarse, limiting y-axis parallelism.

On Firefly Server (P=128 threads, for `images/place_23k_23k.png`, see Figure 4): The trend is similar:

- Small T (e.g., $T = 1$): Low speed-up ($3.77\times$) due to high overhead.
- Increasing T: Speed-up improves, peaking at $T = 200$ ($28.08\times$).
- Large T (e.g., $T = 500$): Speed-up slightly decreases ($28.03\times$).

On both systems, an optimal threshold T balances task creation overhead with sufficient parallelism. Too small T leads to overhead domination; too large T limits parallelism.

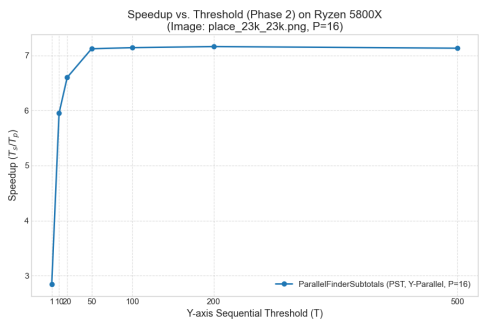

Figure 3: Speedup vs. Threshold (T) for Phase 2 on Ryzen 5800X (P=16, `images/place_23k_23k.png`).

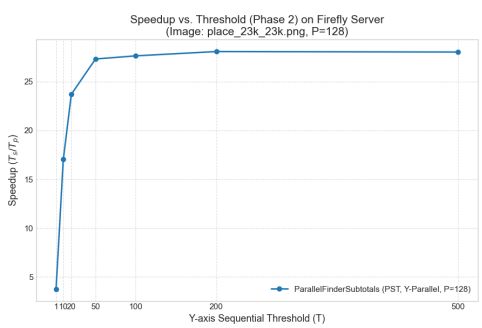

Figure 4: Speedup vs. Threshold (T) for Phase 2 on Firefly Server (P=128, `images/place_23k_23k.png`).

2.3.2 Suppose you would need to determine a threshold naively (i.e., without benchmarking). How would you do this? What would you consider?

Determining a threshold naively involves considering:

1. **Minimum Work per Task:** Estimate task overhead. Work per task should significantly exceed this. If task overhead is equivalent to processing 50-100 rows, T should be above this.
2. **Available Parallelism:** For image height H, and P_y effective threads for y-processing, $T \approx H/(k \cdot P_y)$ where k is a small factor (2-4).
3. **Cache Locality:** Larger T might improve cache utilization.
4. **Problem Size (H):** For $H = 23000$, dividing by a few hundred (e.g., $23000/150 \approx 153$) seems reasonable for task count.

A naive estimate for T might fall in the $100 - 250$ range.

2.3.3 Based on your measurements, what is the best threshold? Why?

On Local Ryzen 5800X: The best threshold observed is $T \approx 200$, yielding a speed-up of $7.16\times$. **On Firefly Server:** The best threshold observed is $T \approx 200$, yielding a speed-up of $28.08\times$.

Interestingly, $T \approx 200$ appears robustly optimal for the `images/place_23k_23k.png` dataset across both architectures. This suggests an intrinsic task granularity sweet spot for the y-axis processing of this specific image data, somewhat independent of the total core count (once sufficient x-tasks are also running). It provides the best balance between exploiting parallelism and managing task overhead.

2.4 Implementation Characteristics (`ParallelFinderSubtotals` - Phase 1 & 2)

This theoretical analysis focuses on my `ParallelFinderSubtotals` implementation. Let W denote image width and H the image height. S_X is the x-axis threshold (100), and T is the y-axis threshold. Merging HashMaps is $O(1)$. `Amongus.detect` is $O(1)$ per pixel.

2.4.1 Work (T_1)

$T_1 = O(W \times H)$.

2.4.2 Span (T_∞)

- **Phase 1 (PST, Y sequential):** $T_\infty(\text{Phase 1}) = O(H + \log(W/S_X))$.
- **Phase 2 (PST, Y parallel):** $T_\infty(\text{Phase 2}) = O(T + \log(H/T) + \log(W/S_X))$.

2.4.3 Number of Tasks Created

- **`XFinderTasks`:** Approximately $O(W/S_X)$.
- **`YFinderTasks` (Phase 2):** Approximately $O(W \times H/T)$.
- **Total tasks (Phase 2):** $\approx O(W/S_X + WH/T)$.

The number of Y-tasks can be very large if T is small, contributing to overhead.

3 Conclusion

In this project, I successfully applied the Java Fork/Join framework to parallelize the computationally intensive task of detecting Amongi characters in large images. My evaluation on both my local Ryzen 5800X system and the Firefly server demonstrated that both Phase 1 parallelization strategies yield substantial speed-ups. The local HashMap (subtotals) method generally showed slightly lower or comparable overhead and similar scalability. On the Ryzen 5800X (8c/16t via SMT), a peak speed-up of $\approx 7.25\times$ was achieved with 16 threads. On the Firefly server (64c/128t via SMT), a peak speed-up of $\approx 31\times$ was achieved with 64 cores. Performance

degraded when utilizing all logical threads beyond physical cores on both systems, highlighting the limitations of SMT for this workload and the impact of increased contention.
The Phase 2 implementation (y-axis parallelism combined with x-axis parallelism using the subtotals method) further improved performance. The optimal y-axis sequential threshold T was found to be around 200 for the largest test image on both systems, achieving a speed-up of $\approx 7.16\times$ on the Ryzen 5800X (16 threads) and $\approx 28.08\times$ on Firefly (128 threads). This consistency in optimal T suggests an intrinsic task granularity sweet spot for the y-axis processing of this specific image data. The project underscores the critical role of careful parameter tuning and understanding hardware characteristics in designing efficient parallel applications. The theoretical analysis of work, span, and task creation provided a framework for understanding the observed performance.

References

[1] Goetz, B., Peierls, T., Bloch, J., Bowbeer, J., Lea, D., & Holmes, D. (2006). *Java Concurrency in Practice*. Addison-Wesley Professional.

[2] Lea, D. (1999). *Concurrent Programming in Java: Design Principles and Patterns* (2nd ed.). Addison-Wesley Professional.

[3] Oracle Corporation. (n.d.). *Fork/Join*. Java Platform, Standard Edition 8, API Specification. Retrieved May 22, 2025, from https://docs.oracle.com/javase/8/docs/api/java/util/concurrent/ForkJoinPool.html

[4] Jain, R. (1991). *The Art of Computer Systems Performance Analysis: Techniques for Experimental Design, Measurement, Simulation, and Modeling*. Wiley.

[5] Amdahl, G. M. (1967). Validity of the single processor approach to achieving large scale computing capabilities. *AFIPS Conference Proceedings, 30*, 483-485.

[6] Herlihy, M., Shavit, N., Luchangco, V., & Spear, M. (2020). *The Art of Multiprocessor Programming* (2nd ed.). Morgan Kaufmann.

[7] Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). *Introduction to Algorithms* (3rd ed.). MIT Press.

[8] Gonzalez Boix, E., & Van der Plas, J. (2022). *Guidelines for Performance Evaluation*. Vrije Universiteit Brussel, Parallelism and Distribution Course.

[9] Silberschatz, A., Galvin, P. B., & Gagne, G. (2018). *Operating System Concepts* (10th ed.). Wiley.